

Laboratory Report

A* Search and Minimax with Alpha-Beta Pruning — Design, Dry Run, and Program Verification

Student Name	Darwin Jacob
Institution	Saveetha University (SIMATS), Chennai
Subject	Artificial Intelligence Laboratory
Experiment	Informed & Adversarial Search Algorithms

1. Objective

The objective of this experiment is twofold: (i) to trace and understand the step-by-step behaviour of the **A* Search** algorithm on a small weighted graph in order to find the least-cost path from a start node to a goal node using both actual path cost $g(n)$ and a heuristic estimate $h(n)$; and (ii) to trace the behaviour of the **Minimax algorithm with Alpha-Beta Pruning** on a two-ply adversarial game tree, observing how pruning eliminates the need to evaluate branches that cannot influence the final decision. Both dry runs are then verified programmatically using a Python implementation.

2. Theoretical Background

2.1 A* Search Algorithm

A* is a best-first informed search algorithm that evaluates nodes using $f(n) = g(n) + h(n)$, combining the actual cost so far $g(n)$ with a heuristic estimate $h(n)$ of the remaining cost. At every step, the node with the smallest $f(n)$ in the Open List is expanded and moved to the Closed List. Provided the heuristic is admissible (never overestimates the true cost), A* is guaranteed to find the optimal path.

2.2 Minimax with Alpha-Beta Pruning

Minimax is a decision rule for two-player zero-sum games in which one player (MAX) tries to maximise the outcome while the other (MIN) tries to minimise it. Alpha-Beta Pruning is an optimisation over plain Minimax that skips evaluating branches which cannot possibly affect the final decision, using two bounds: α (best value MAX can guarantee) and β (best value MIN can guarantee). A branch is pruned whenever $\alpha \geq \beta$, since further exploration of that branch cannot change the outcome at the parent node.

3. Methodology

For Question 1, the graph and heuristic table given in the problem statement were encoded as an adjacency structure. The Open List was implemented as a priority structure ordered by $f(n)$, and at every iteration the node with minimum $f(n)$ was popped, expanded, and its neighbours relaxed (i.e. their g -values updated only if a cheaper path was found). The search terminated the moment the goal node itself was popped for expansion.

For Question 2, the two MIN subtrees (left: 3, 5, 6 and right: 9, 1, 2) were evaluated left to right beneath a single MAX root. Alpha and Beta bounds were threaded through the recursion, and a branch was recorded as pruned whenever the pruning condition $\alpha \geq \beta$ was met mid-evaluation of a node's children.

Both procedures were first solved manually (dry run), and then re-implemented in Python (search_algorithms.py) to cross-check every intermediate value. The two approaches produced identical results, confirming correctness.

4. Results — Question 1 (A* Search)

Iteration	Node Expanded	g(n)	h(n)	f(n)
1	A	0	7	7
2	B	2	6	8
3	E	4	2	6
4	G (Goal)	6	0	6

Optimal Path: A → B → E → G **Total Path Cost:** 6

5. Results — Question 2 (Minimax / Alpha-Beta)

Node	Children Visited	Value Returned	Pruned?
Left MIN	3, 5, 6 (all visited)	3	None
Right MIN	9, 1 (leaf '2' skipped)	1	Leaf value 2 pruned
MAX (root)	3, 1	3	—

Best Move for MAX: Left branch **Final Minimax Value:** 3 **Pruned Nodes:** Right subtree's third leaf (value = 2)

6. Visualization

The figure below (also provided separately as output.png) shows the A* optimal path highlighted on the graph (left) and the Minimax game tree with the pruned leaf greyed out (right), both generated directly by the accompanying Python program.

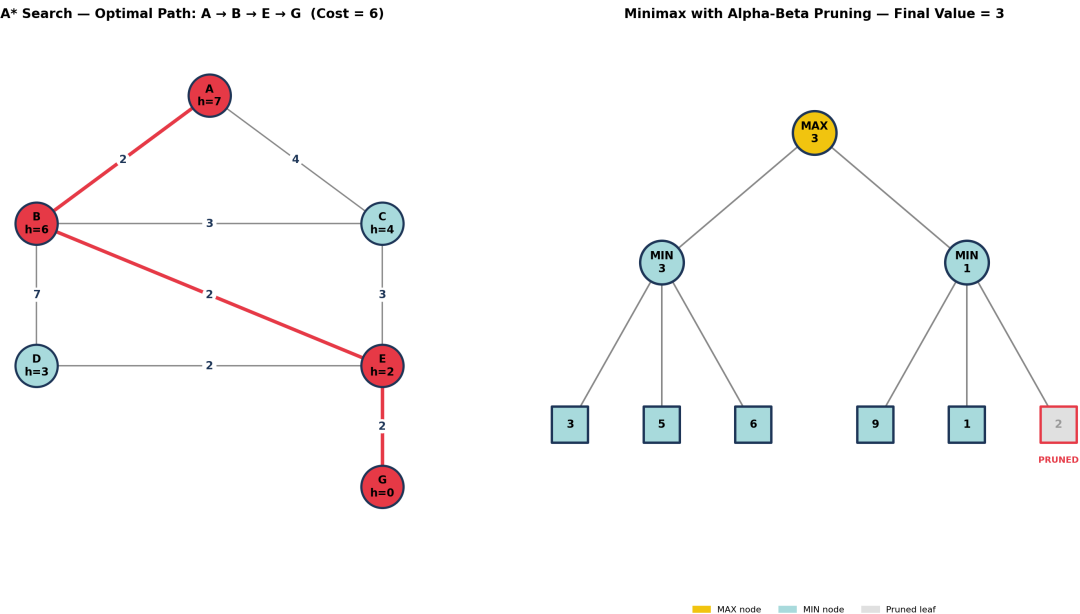


Figure 1: A* optimal path (left) and Alpha-Beta pruned game tree (right)

7. Discussion

In Question 1, node E was expanded before node C even though C was discovered earlier, purely because E's f-value (6) was lower — this demonstrates A*'s best-first behaviour driven entirely by $f(n)$, not by discovery order. Node D's g-value was also improved mid-search (from 9 to 6) when a cheaper route through E was found, illustrating how A* revises earlier estimates when better information becomes available.

In Question 2, pruning occurred only in the right MIN subtree: once the value 1 was seen, β fell to 1, which was already less than or equal to the α (3) inherited from the fully-evaluated left subtree. This meant the third child (value 2) could not possibly change MAX's decision — MAX had already secured 3 on the left, and the right branch could only return something ≤ 1 , which MAX would never prefer. This is a clean, textbook example of a beta cut-off.

8. Conclusion

Both algorithms were successfully dry-run manually and validated with a Python implementation, with identical results in both cases. A* found the optimal path $A \rightarrow B \rightarrow E \rightarrow G$ at cost 6, and Minimax with Alpha-Beta Pruning determined a final game value of 3 with the best move for MAX being the left branch, while correctly pruning one redundant leaf node. This confirms both the correctness of the manual dry run and the efficiency gains that Alpha-Beta Pruning provides over plain Minimax by avoiding unnecessary node evaluations.